LAPORAN AKHIR TUGAS BESAR

IMPLEMENTASI KEAMANAN SERVER DAN CONTAINERISASI WEB APPLICATION MENGGUNAKAN NGINX REVERSE PROXY DAN DOCKER

Dosen Pengampu : Ainul Hizriadi S.Kom., M.Sc



DISUSUN OLEH:

KOM C - KELOMPOK 4

Ilma Mahmudah	241402003
Putri Sahara Tampubolon	241402015
Anggun Dwikasih Mahrani Harahap	241402018
Rasyiqah Amalina Putri	241402024
Dhaifina Raiqa Zahira	241402056
Nazwa Sahara Harahap	241402126
Rahmaini Harna	241402132

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI

UNIVERSITAS SUMATERA UTARA

2026

DAFTAR ISI

DAFTAR ISI 2
DAFTAR GAMBAR 4
BAB I PENDAHULUAN 5
1.1 Latar Belakang 5
1.2 Rumusan Masalah 6
1.3 Tujuan 6
1.4 Manfaat 6
BAB II LANDASAN TEORI 7
2.1 Linux Server 7
2.2 Docker 7
2.3 Docker Compose 7
2.4 Nginx Reverse Proxy 8
2.5 WordPress 8
2.6 MariaDB 8
2.7 Firewall UFW 8
2.8 HTTPS dan SSL/TLS 8
2.9 Fail2Ban 9
2.10 SSH Hardening 9
BAB III PERANCANGAN SISTEM 10
3.1 Deskripsi Sistem 10
3.2 Arsitektur Sistem 10
3.3 Struktur Container 11
3.4 Perancangan Keamanan 12
3.5 Perancangan Docker Network 13
BAB IV IMPLEMENTASI SISTEM 14
4.1 Persiapan Environment 14
4.2 Pembuatan Struktur Project 15
4.3 Konfigurasi Docker Compose 15
4.4 Konfigurasi Firewall UFW 16
4.5 Git Repository 17
4.6 Konfigurasi SSH Hardening 18

4.7 Implementasi HTTPS dan SSL 19
4.8 Implementasi Fail2Ban 20
4.9 Hasil Running Container 21
4.10 Implementasi WordPress 22
BAB V PENGUJIAN DAN ANALISIS 24
5.1 Pengujian Sistem 24
5.2 Pengujian Keamanan 25
5.3 Analisis Hasil 25
BAB VI PENUTUP 26
6.1 Kesimpulan 26
6.2 Saran 26
DAFTAR PUSTAKA 28

DAFTAR GAMBAR

Gambar 1 Arsitektur Sistem 11
Gambar 2 Proses Update Sistem Ubuntu 14
Gambar 3 Instalasi Docker pada Ubuntu Server 14
Gambar 4 Verifikasi Docker Compose 15
Gambar 5 Pembuatan Direktori Project 15
Gambar 6 Proses Deployment Container Menggunakan Docker Compose 16
Gambar 7 Instalasi Firewall UFW pada Ubuntu Server 16
Gambar 8 Konfigurasi Allow Port pada Firewall UFW 17
Gambar 9 Aktivasi Firewall UFW 17
Gambar 10 Hasil Verifikasi Konfigurasi Firewall UFW pada Ubuntu Server 17
Gambar 11 Inisialisasi Git Repository pada Project Docker 18
Gambar 12 Konfigurasi SSH Hardening pada Ubuntu Server 19
Gambar 13 Implementasi HTTPS pada Website WordPress 20
Gambar 14 Instalasi Fail2Ban pada Ubuntu Server 20
Gambar 15 Aktivasi Service Fail2Ban 21
Gambar 16 Verifikasi Status Fail2Ban pada Ubuntu Server 21
Gambar 17 Hasil Running Container Docker 21
Gambar 18 Halaman Instalasi WordPress 22
Gambar 19 Hasil Instalasi WordPress Berhasil 23
Gambar 20 Dashboard Administrator WordPress 23

BAB I
PENDAHULUAN

1.1 Latar Belakang

Menurut (Dwiyatno et al., 2020), perkembangan aplikasi berbasis web saat ini semakin pesat sehingga proses deployment aplikasi beserta software pendukung seperti web server, database server, dependency, dan environment lainnya membutuhkan pengelolaan yang lebih efisien. Pada sistem tradisional, beberapa aplikasi web dijalankan dalam satu server yang sama sehingga dapat menimbulkan masalah seperti konflik dependency, tingginya penggunaan resource, dan rendahnya tingkat isolasi antar aplikasi.

Menurut (Adiputra Firmansyah, 2015), teknologi virtualisasi berbasis container menjadi solusi untuk mengatasi permasalahan tersebut karena mampu menyediakan lingkungan aplikasi yang terisolasi dengan penggunaan resource yang lebih ringan dibandingkan virtual machine berbasis hypervisor. Docker sebagai platform containerisasi memungkinkan aplikasi dijalankan secara cepat, portabel, dan efisien tanpa harus menjalankan guest operating system secara terpisah pada setiap layanan.

Selain itu, (Dwiyatno et al., 2020) menjelaskan bahwa penggunaan Nginx sebagai reverse proxy dapat membantu mengatur lalu lintas request dari client menuju server aplikasi serta meningkatkan efisiensi penggunaan resource server. Reverse proxy juga membantu menyembunyikan alamat internal server dan mendukung komunikasi antar container dalam satu Docker network.

Di sisi lain, keamanan server menjadi aspek penting dalam pengelolaan layanan web modern. Menurut (Rakhmadi Rahman et al., 2026), server Linux dengan konfigurasi default masih rentan terhadap berbagai ancaman siber seperti brute force attack, banyaknya port terbuka yang tidak diperlukan, serta penggunaan autentikasi password yang lemah pada layanan SSH. Oleh karena itu diperlukan implementasi hardening server menggunakan mekanisme keamanan seperti Firewall UFW, SSH Hardening, HTTPS/SSL, dan Fail2Ban guna mengurangi attack surface dan meningkatkan keamanan sistem server.

Berdasarkan permasalahan tersebut, pada proyek ini dilakukan implementasi layanan web berbasis Docker menggunakan arsitektur tiga container yang terdiri dari Nginx sebagai reverse proxy, WordPress sebagai web application, dan MariaDB sebagai database server.

Selain itu diterapkan beberapa mekanisme keamanan server Linux untuk meningkatkan keamanan dan stabilitas layanan web.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah pada proyek ini adalah sebagai berikut:

1. Bagaimana mengimplementasikan virtualisasi server berbasis Docker container untuk layanan web?
2. Bagaimana menerapkan Nginx sebagai reverse proxy pada arsitektur container?
3. Bagaimana mengimplementasikan keamanan server Linux menggunakan UFW, SSH Hardening, HTTPS, dan Fail2Ban?
4. Bagaimana membangun layanan web yang stabil dan efisien menggunakan Docker container?

1.3 Tujuan

Berdasarkan rumusan masalah yang telah dijelaskan, maka tujuan dari proyek ini adalah sebagai berikut:

1. Mengimplementasikan layanan web berbasis Docker container.
2. Mengimplementasikan Nginx sebagai reverse proxy.
3. Mengimplementasikan WordPress dan MariaDB pada lingkungan container.
4. Menerapkan HTTPS menggunakan SSL/TLS.
5. Mengimplementasikan hardening server Linux menggunakan UFW dan Fail2Ban. Mengisolasi layanan aplikasi menggunakan Docker network.

1.4 Manfaat

Adapun manfaat yang diperoleh dari pelaksanaan proyek ini adalah sebagai berikut:

1. Memahami konsep virtualisasi berbasis container menggunakan Docker.
2. Memahami implementasi reverse proxy menggunakan Nginx.
3. Memahami implementasi keamanan server Linux.
4. Mengetahui proses deployment aplikasi web berbasis container.
5. Mengetahui penerapan hardening server untuk meningkatkan keamanan layanan web.

BAB II
LANDASAN TEORI

2.1 Linux Server

Menurut (Dwiyatno et al., 2020), Linux merupakan sistem operasi berbasis open source yang banyak digunakan pada server karena mampu mendukung berbagai layanan jaringan dan deployment aplikasi secara efisien. Linux juga banyak digunakan pada infrastruktur cloud computing dan web server karena memiliki stabilitas, fleksibilitas, dan performa yang baik dalam pengelolaan layanan server.

Selain itu, (Rakhmadi Rahman et al., 2026) menjelaskan bahwa Linux mendukung implementasi hardening server melalui pengaturan firewall, autentikasi SSH, monitoring log, dan pembatasan layanan yang berjalan pada server untuk meningkatkan keamanan sistem.

2.2 Docker

Menurut (Adiputra Firmansyah, 2015), Docker merupakan platform containerisasi yang digunakan untuk menjalankan aplikasi pada lingkungan yang terisolasi tanpa perlu menjalankan guest operating system secara terpisah pada setiap layanan. Docker mempermudah proses deployment aplikasi web beserta dependency dan software pendukung lainnya.

Selain itu, (Dwiyatno et al., 2020) menjelaskan bahwa Docker memiliki penggunaan resource yang lebih ringan dibandingkan virtual machine berbasis hypervisor karena container menggunakan kernel sistem operasi yang sama. Docker juga memiliki proses startup yang lebih cepat serta mendukung portabilitas aplikasi pada berbagai lingkungan deployment.

2.3 Docker Compose

Menurut (Adiputra Firmansyah, 2015), teknologi container pada Docker memungkinkan beberapa layanan aplikasi dijalankan secara terisolasi namun tetap dapat saling terhubung dalam satu sistem server. Oleh karena itu Docker Compose digunakan untuk mengelola beberapa container sekaligus dalam satu file konfigurasi sehingga proses deployment aplikasi multi-container menjadi lebih mudah dan efisien.

2.4 Nginx Reverse Proxy

Menurut (Dwiyatno et al., 2020), Nginx merupakan web server dan reverse proxy yang memiliki performa tinggi dalam menangani request client. Nginx sebagai reverse proxy bekerja dengan menerima request dari pengguna kemudian meneruskannya ke server aplikasi yang berada di dalam container.

Selain membantu pengelolaan lalu lintas jaringan, penggunaan reverse proxy juga dapat meningkatkan keamanan dengan menyembunyikan alamat internal server serta membantu implementasi HTTPS pada layanan web berbasis container.

2.5 WordPress

Menurut (Dwiyatno et al., 2020), implementasi container pada aplikasi web dapat mempermudah deployment serta pengelolaan dependency aplikasi pada server. Salah satu aplikasi web yang dapat dijalankan menggunakan container adalah WordPress, yaitu Content Management System (CMS) yang digunakan untuk membangun dan mengelola website secara dinamis.

2.6 MariaDB

Menurut (Adiputra Firmansyah, 2015), penggunaan container memungkinkan setiap layanan aplikasi berjalan secara terisolasi sehingga pengelolaan database menjadi lebih aman dan efisien. Dalam implementasi layanan web, MariaDB digunakan sebagai sistem manajemen basis data relasional untuk menyimpan data aplikasi WordPress seperti data pengguna, artikel, dan konfigurasi website.

2.7 Firewall UFW

Menurut (Rakhmadi Rahman et al., 2026), UFW (Uncomplicated Firewall) merupakan salah satu mekanisme keamanan yang digunakan untuk menerapkan default-deny policy pada server Linux sehingga hanya port tertentu yang dapat diakses dari luar server. Penggunaan UFW membantu mengurangi risiko serangan jaringan dengan membatasi akses ke port yang benar-benar diperlukan seperti port SSH, HTTP, dan HTTPS.

2.8 HTTPS dan SSL/TLS

Menurut (Rakhmadi Rahman et al., 2026), implementasi keamanan server Linux dilakukan dengan menerapkan berbagai mekanisme keamanan seperti firewall, SSH hardening, dan HTTPS untuk mengurangi risiko serangan siber pada server. HTTPS sendiri

merupakan protokol komunikasi jaringan yang menggunakan SSL/TLS untuk mengenkripsi data antara client dan server sehingga komunikasi menjadi lebih aman.

2.9 Fail2Ban

Menurut (Rakhmadi Rahman et al., 2026), Fail2Ban merupakan sistem keamanan yang digunakan untuk mencegah brute force attack dengan memonitor log autentikasi server dan memblokir alamat IP yang melakukan percobaan login berulang. Fail2Ban bekerja dengan menambahkan aturan firewall secara otomatis untuk memblokir IP penyerang. Penggunaan Fail2Ban dapat membantu meningkatkan keamanan layanan SSH dan mengurangi risiko akses tidak sah ke server.

2.10 SSH Hardening

Menurut (Rakhmadi Rahman et al., 2026) , SSH Hardening merupakan proses pengamanan layanan SSH untuk mengurangi risiko serangan terhadap server Linux. Implementasi SSH Hardening dilakukan melalui beberapa teknik seperti mengganti port default SSH, menonaktifkan login root, membatasi jumlah percobaan login, dan menggunakan autentikasi berbasis SSH key. Penerapan SSH Hardening membantu meningkatkan keamanan akses remote server dan meminimalkan risiko brute force attack terhadap layanan SSH.

BAB III
PERANCANGAN SISTEM

3.1 Deskripsi Sistem

Sistem yang dibangun merupakan layanan web berbasis Docker dengan arsitektur multi-container. Sistem ini terdiri dari tiga container utama yaitu Nginx sebagai reverse proxy, WordPress sebagai web application, dan MariaDB sebagai database server. Seluruh layanan dijalankan menggunakan Docker Compose sehingga proses deployment dan pengelolaan container menjadi lebih mudah.

Container WordPress digunakan untuk menjalankan aplikasi website, sedangkan MariaDB digunakan sebagai media penyimpanan data aplikasi. Nginx berfungsi sebagai reverse proxy yang menerima request dari client kemudian meneruskannya ke layanan WordPress. Seluruh container dihubungkan menggunakan Docker network agar komunikasi antar layanan dapat berjalan dengan baik dan lebih terisolasi.

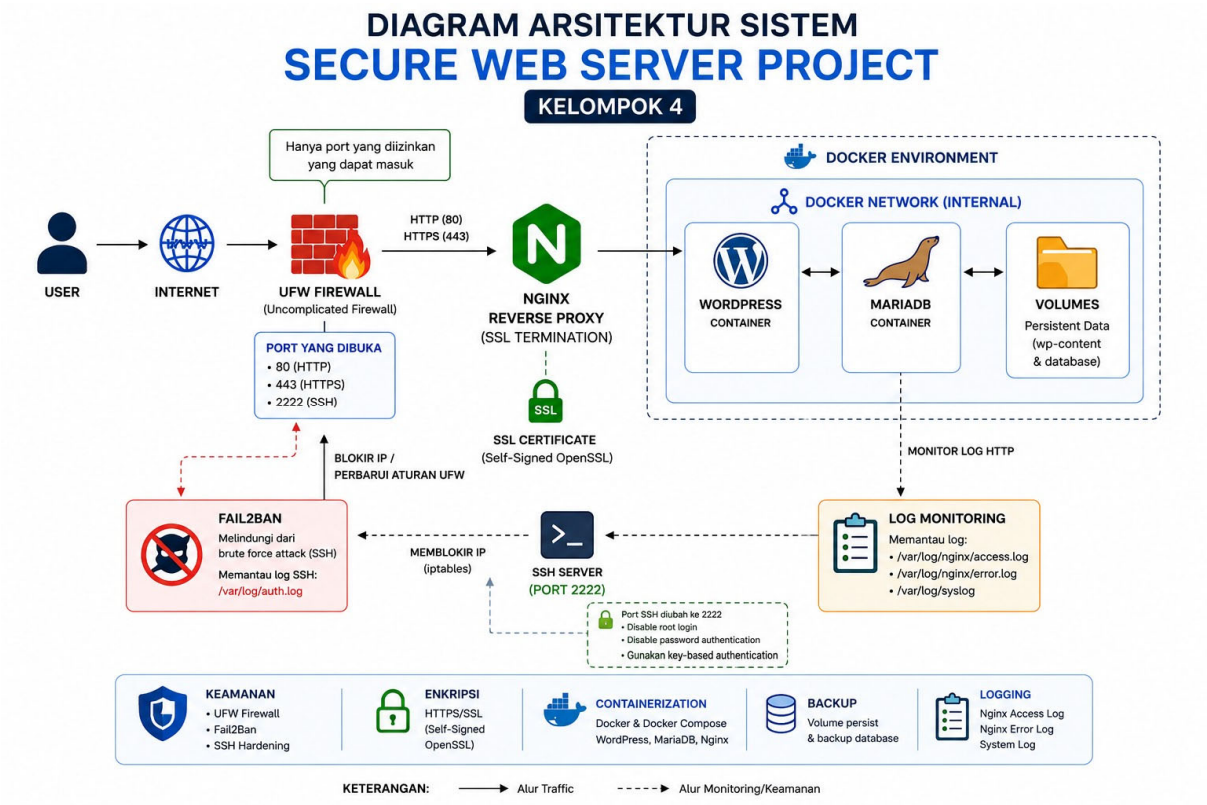
3.2 Arsitektur Sistem

Arsitektur sistem yang dibangun menggunakan model client-server berbasis Docker container dengan pendekatan multi-container. Sistem terdiri dari layanan utama yaitu Nginx sebagai reverse proxy, WordPress sebagai web application, dan MariaDB sebagai database server. Selain itu sistem juga menerapkan beberapa mekanisme keamanan seperti Firewall UFW, HTTPS/SSL, SSH Hardening, dan Fail2Ban untuk meningkatkan keamanan layanan web.

Pada arsitektur ini, user mengakses website melalui browser menggunakan protokol HTTP maupun HTTPS melalui internet. Request dari client akan melewati Firewall UFW yang berfungsi membatasi akses jaringan sehingga hanya port tertentu yang dibuka yaitu port 80 untuk HTTP, port 443 untuk HTTPS, dan port 2222 untuk SSH. Selanjutnya request diteruskan ke Nginx reverse proxy sebelum diproses oleh aplikasi WordPress yang terhubung dengan MariaDB melalui Docker internal network.

Layanan WordPress dan MariaDB dijalankan menggunakan Docker container yang saling terhubung melalui Docker network sehingga komunikasi antar layanan menjadi lebih aman dan terisolasi. Sistem keamanan juga diperkuat menggunakan HTTPS/SSL dengan sertifikat self-signed OpenSSL untuk mengenkripsi komunikasi data, sedangkan Fail2Ban

digunakan untuk memonitor layanan SSH dan membantu mencegah brute force attack pada server.



Gambar 1 Arsitektur Sistem

Diagram di atas menunjukkan alur komunikasi dan mekanisme keamanan pada sistem secure web server yang dibangun. User mengakses layanan website melalui internet kemudian request akan difilter oleh Firewall UFW sebelum diteruskan ke layanan Nginx reverse proxy. Selanjutnya request diproses oleh aplikasi WordPress yang terhubung dengan MariaDB melalui Docker internal network. Selain implementasi layanan web, sistem juga dilengkapi dengan mekanisme keamanan seperti HTTPS/SSL, Fail2Ban, dan SSH Hardening untuk meningkatkan keamanan server dan melindungi layanan dari ancaman jaringan.

3.3 Struktur Container

Berikut merupakan struktur container yang digunakan pada sistem:

Container	Fungsi
nginx	Reverse Proxy
wordpress	Web Application
mariadb	Database Server

Tabel 1 Struktur Container

Setiap container memiliki fungsi yang berbeda sehingga layanan dapat berjalan secara terpisah namun tetap saling terhubung melalui Docker network. Container Nginx berfungsi sebagai reverse proxy yang menerima seluruh request dari client sebelum diteruskan ke aplikasi WordPress. Container WordPress digunakan untuk menjalankan website dan memproses seluruh aktivitas pengguna, sedangkan container MariaDB berfungsi sebagai media penyimpanan data aplikasi website.

Pemisahan layanan ke dalam beberapa container memberikan keuntungan dalam hal isolasi sistem dan pengelolaan aplikasi. Dengan arsitektur ini, setiap layanan dapat dikonfigurasi, dijalankan, maupun diperbarui secara mandiri tanpa mempengaruhi layanan lainnya. Selain itu proses troubleshooting menjadi lebih mudah karena setiap container memiliki fungsi yang spesifik dan terpisah.

Penggunaan Docker network juga membantu meningkatkan keamanan dan efisiensi komunikasi antar layanan. Container database tidak perlu diekspos langsung ke host server karena hanya dapat diakses oleh container WordPress melalui jaringan internal Docker. Dengan demikian, risiko akses tidak sah terhadap database dapat dikurangi dan pengelolaan layanan menjadi lebih aman serta terstruktur.

3.4 Perancangan Keamanan

Perancangan keamanan server dilakukan untuk meningkatkan keamanan layanan web dan meminimalkan risiko serangan terhadap server. Mekanisme keamanan yang diterapkan pada sistem meliputi:

1. Firewall UFW dengan konfigurasi default deny incoming untuk membatasi akses jaringan yang tidak diperlukan.
2. SSH Hardening menggunakan autentikasi berbasis SSH key dan pengamanan akses remote server.
3. Fail2Ban untuk mencegah brute force attack dengan memblokir alamat IP yang melakukan percobaan login berulang.
4. Database container tidak diekspos langsung ke host sehingga akses database hanya dapat dilakukan melalui container yang berada dalam Docker network.

3.5 Perancangan Docker Network

Sistem yang dibangun menggunakan Docker bridge network untuk menghubungkan seluruh container yang berjalan. Docker network memungkinkan komunikasi antar container dapat berlangsung dengan baik tanpa harus mengekspos seluruh layanan secara langsung ke host server. Dengan menggunakan jaringan internal Docker, setiap container dapat saling bertukar data dan menjalankan layanan sesuai fungsinya masing-masing secara lebih aman dan terstruktur.

Pada implementasi ini, container Nginx, WordPress, dan MariaDB berada pada satu jaringan Docker yang sama sehingga komunikasi antar layanan dapat dilakukan menggunakan nama service atau nama container. Nginx berkomunikasi dengan WordPress untuk meneruskan request dari client, sedangkan WordPress berkomunikasi dengan MariaDB untuk melakukan proses penyimpanan dan pengambilan data website. Penggunaan Docker network membantu mempermudah konfigurasi koneksi antar container tanpa perlu menggunakan alamat IP secara manual.

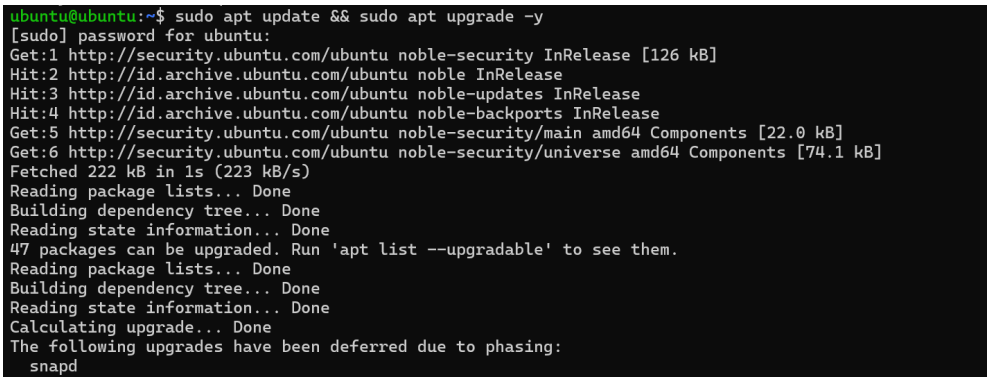
Selain mempermudah komunikasi antar layanan, Docker bridge network juga memberikan keuntungan dalam hal keamanan dan isolasi sistem. Container database tidak perlu diekspos langsung ke host server karena hanya dapat diakses melalui jaringan internal Docker oleh container WordPress. Dengan demikian, risiko akses tidak sah terhadap database dapat dikurangi dan keamanan layanan menjadi lebih baik.

Penggunaan Docker network juga membantu proses pengelolaan dan pengembangan sistem di masa mendatang. Apabila terdapat penambahan layanan baru seperti monitoring server atau load balancer, container tambahan dapat dengan mudah dihubungkan ke jaringan Docker yang sudah ada tanpa mengubah keseluruhan arsitektur sistem. Oleh karena itu, penggunaan Docker bridge network menjadi solusi yang efektif untuk membangun layanan web berbasis container yang aman, fleksibel, dan mudah dikembangkan.

BAB IV
IMPLEMENTASI SISTEM

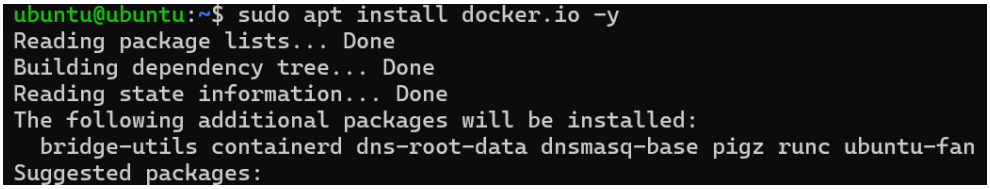
4.1 Persiapan Environment

Tahap awal implementasi dilakukan dengan melakukan update package sistem Ubuntu untuk memastikan seluruh package berada pada versi terbaru. Setelah itu dilakukan instalasi Docker sebagai platform containerisasi yang digunakan untuk menjalankan layanan aplikasi berbasis container. Perintah yang digunakan untuk melakukan update sistem adalah sebagai berikut:



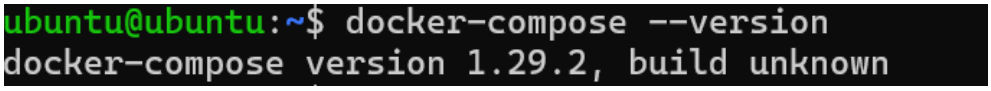
Gambar 2 Proses Update Sistem Ubuntu

Berdasarkan hasil implementasi, proses update package berhasil dilakukan dan sistem berhasil mengambil repository terbaru dari Ubuntu Server. Selanjutnya dilakukan instalasi Docker menggunakan perintah berikut:



Gambar 3 Instalasi Docker pada Ubuntu Server

Hasil instalasi menunjukkan Docker berhasil dipasang pada sistem Ubuntu beserta dependency yang dibutuhkan untuk menjalankan layanan container. Setelah Docker berhasil diinstal, dilakukan pengecekan versi Docker Compose untuk memastikan layanan Docker Compose telah tersedia pada sistem.

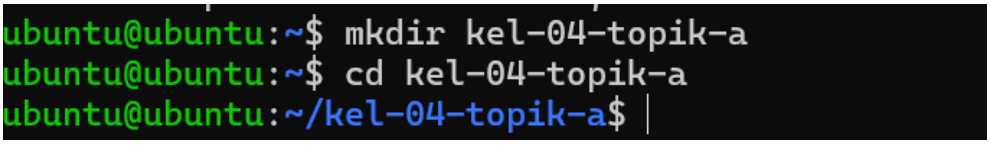


Gambar 4 Verifikasi Docker Compose

Berdasarkan hasil pengujian, sistem berhasil menampilkan informasi versi Docker Compose yaitu versi 1.29.2. Hasil tersebut menunjukkan bahwa Docker Compose telah berhasil diinstal dan siap digunakan untuk mengelola layanan multi-container pada proyek yang dibangun.

4.2 Pembuatan Struktur Project

Selanjutnya dilakukan pembuatan direktori project yang digunakan untuk menyimpan file konfigurasi Docker dan layanan website yang akan dijalankan.

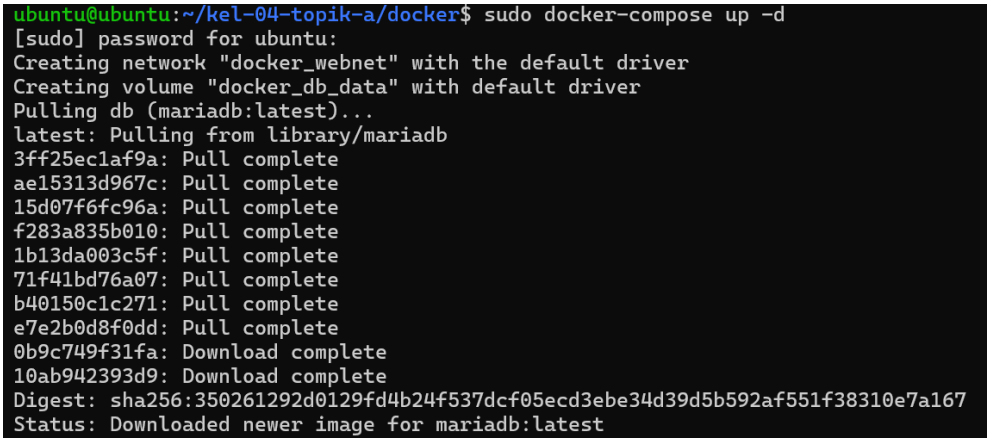


Gambar 5 Pembuatan Direktori Project

Direktori project berhasil dibuat dan digunakan sebagai workspace implementasi layanan web berbasis Docker container. Struktur direktori ini membantu mempermudah pengelolaan file konfigurasi seperti docker-compose.yml, konfigurasi Nginx, serta file layanan lainnya agar lebih terorganisir dan mudah dikelola selama proses deployment sistem.

4.3 Konfigurasi Docker Compose

Docker Compose digunakan untuk menjalankan layanan multi-container yang terdiri dari Nginx, WordPress, dan MariaDB. Proses deployment container dilakukan menggunakan perintah sebagai berikut:

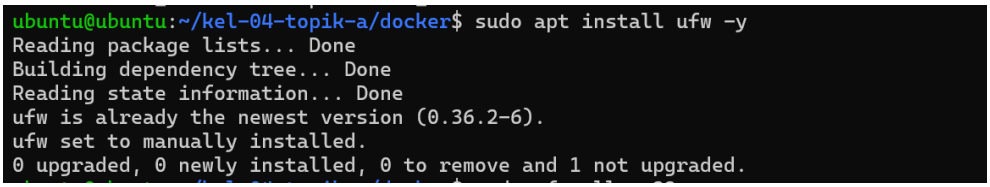


Gambar 6 Proses Deployment Container Menggunakan Docker Compose

Berdasarkan hasil implementasi, Docker berhasil membuat Docker network dan volume yang digunakan untuk komunikasi antar container serta penyimpanan data aplikasi. Selain itu sistem juga berhasil melakukan proses pull image MariaDB dari Docker Hub sebagai bagian dari layanan database yang digunakan pada website WordPress. Hasil tersebut menunjukkan bahwa konfigurasi Docker Compose berhasil dijalankan dan seluruh layanan container dapat dipersiapkan dengan baik sebelum sistem dioperasikan

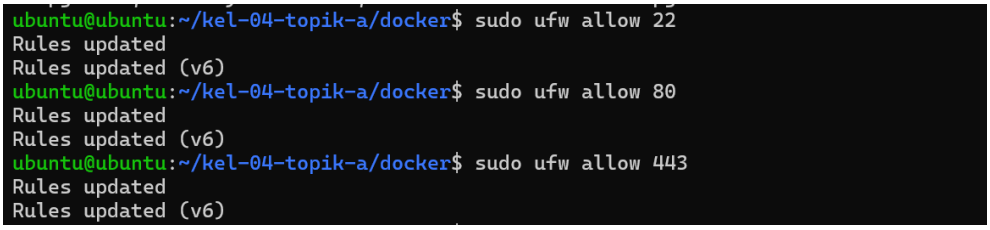
4.4 Konfigurasi Firewall UFW

Firewall UFW (Uncomplicated Firewall) digunakan untuk meningkatkan keamanan server dengan membatasi akses jaringan yang diperbolehkan. Pada tahap ini dilakukan instalasi dan konfigurasi UFW dengan menerapkan policy default deny incoming dan allow outgoing agar server hanya membuka akses pada port tertentu yang dibutuhkan layanan sistem. Perintah yang digunakan untuk instalasi UFW adalah sebagai berikut:



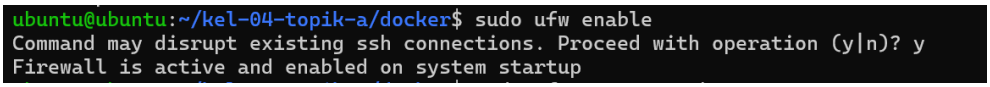
Gambar 7 Instalasi Firewall UFW pada Ubuntu Server

Setelah proses instalasi selesai, dilakukan konfigurasi firewall dengan membuka beberapa port yang dibutuhkan sistem yaitu port 22 untuk layanan SSH, port 80 untuk layanan HTTP, dan port 443 untuk layanan HTTPS. Perintah yang digunakan adalah sebagai berikut:



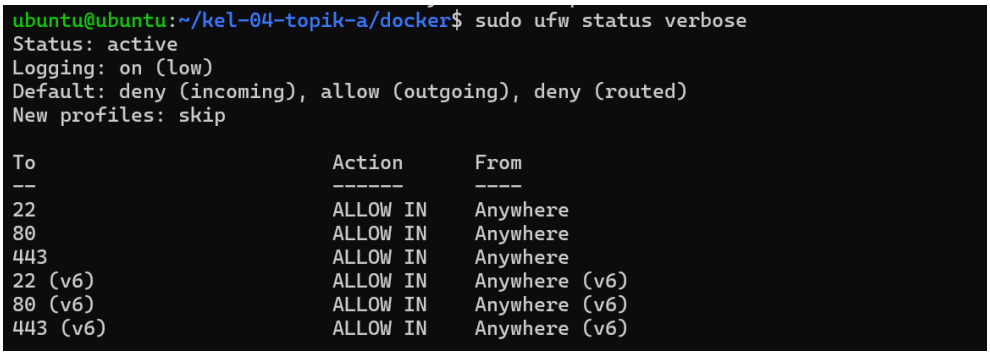
Gambar 8 Konfigurasi Allow Port pada Firewall UFW

Selanjutnya firewall diaktifkan menggunakan perintah berikut:



Gambar 9 Aktivasi Firewall UFW

Hasil implementasi menunjukkan bahwa firewall berhasil aktif pada sistem Ubuntu dan otomatis berjalan saat server dinyalakan. Untuk memastikan konfigurasi firewall berhasil diterapkan, dilakukan pengecekan status UFW menggunakan perintah berikut:

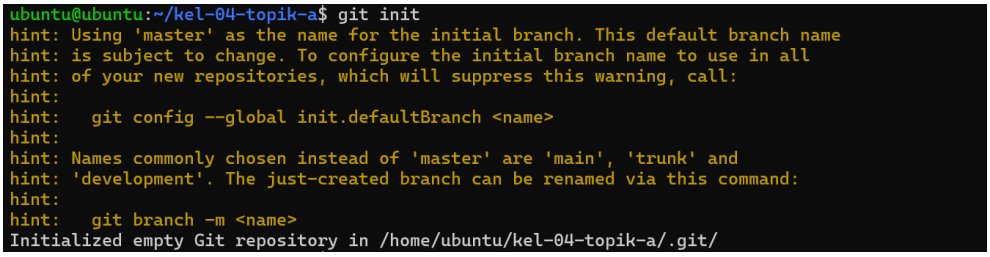


Gambar 10 Hasil Verifikasi Konfigurasi Firewall UFW pada Ubuntu Server

Berdasarkan hasil pengujian, firewall berhasil menerapkan policy keamanan dengan status default deny incoming dan allow outgoing. Selain itu hanya port tertentu yang diizinkan untuk diakses dari luar server yaitu port 22, 80, dan 443.

4.5 Git Repository

Git Repository digunakan untuk menyimpan dan mengelola source code project selama proses pengembangan sistem. Penggunaan Git membantu mempermudah proses version control sehingga setiap perubahan konfigurasi dan source code dapat dikelola dengan lebih terstruktur. Pembuatan repository Git dilakukan menggunakan perintah berikut:



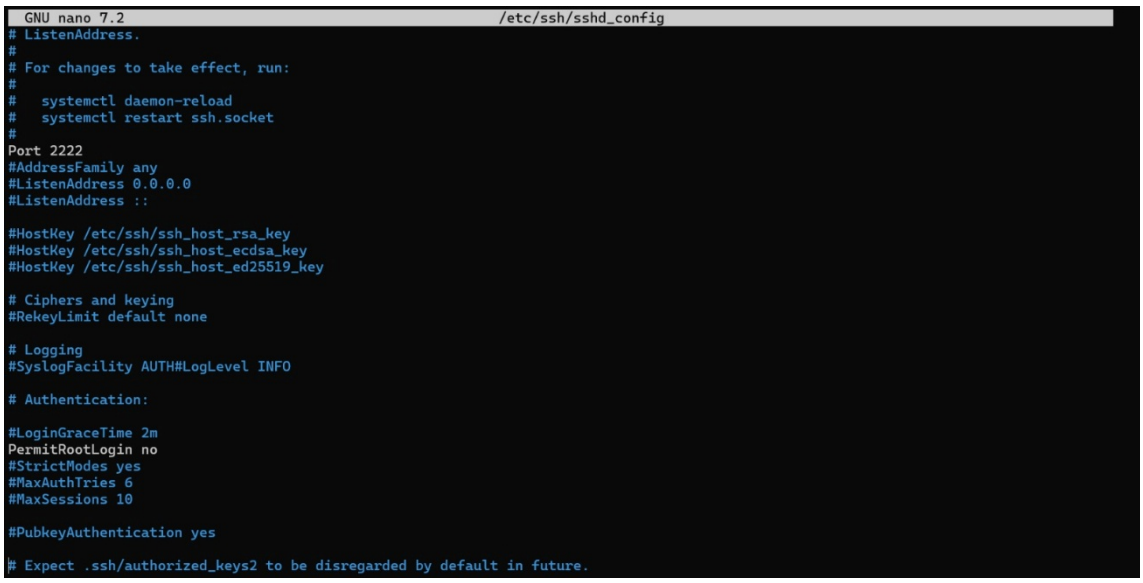
Gambar 11 Inisialisasi Git Repository pada Project Docker

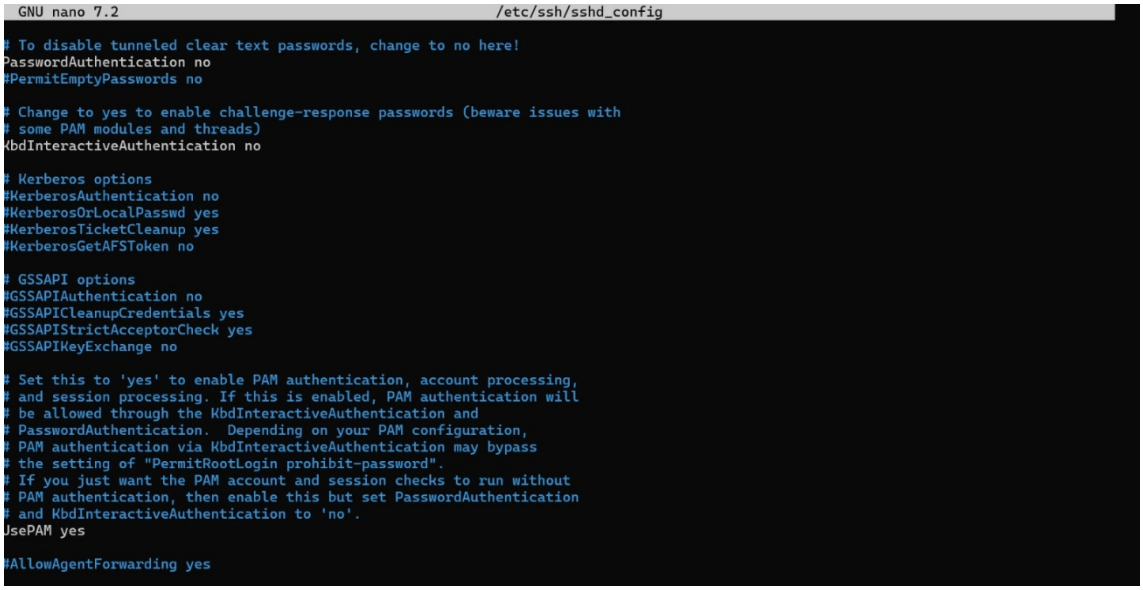
Berdasarkan hasil implementasi, sistem berhasil membuat repository Git baru pada direktori project kel-04-topik-a. Hal ini ditunjukkan dengan munculnya informasi Initialized empty Git repository pada terminal yang menandakan bahwa proses inisialisasi repository berhasil dilakukan.

Repository Git selanjutnya digunakan untuk menyimpan file konfigurasi Docker, konfigurasi layanan web, serta dokumentasi project agar pengelolaan source code menjadi lebih mudah dan terorganisir selama proses pengembangan sistem.

4.6 Konfigurasi SSH Hardening

SSH Hardening dilakukan untuk meningkatkan keamanan akses remote server melalui layanan SSH. Konfigurasi dilakukan dengan melakukan perubahan pada file konfigurasi SSH yang terdapat pada sistem Ubuntu Server.





Gambar 12 Konfigurasi SSH Hardening pada Ubuntu Server

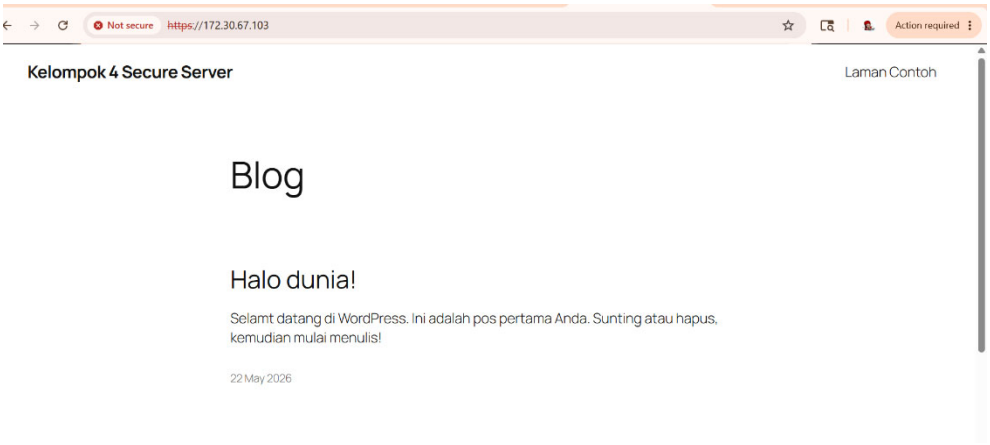
Berdasarkan hasil implementasi, konfigurasi SSH berhasil dilakukan dengan mengubah port default SSH menjadi port 2222, menonaktifkan login root, serta menonaktifkan autentikasi berbasis password. Selain itu sistem dikonfigurasi untuk menggunakan autentikasi berbasis SSH key melalui parameter PubkeyAuthentication yes.

Penerapan SSH Hardening bertujuan untuk mengurangi risiko serangan brute force dan akses tidak sah terhadap server. Dengan menggunakan port non-standar dan autentikasi berbasis SSH key, keamanan layanan SSH menjadi lebih baik dibandingkan konfigurasi default yang menggunakan autentikasi password.

4.7 Implementasi HTTPS dan SSL

HTTPS diterapkan pada layanan website untuk mengenkripsi komunikasi antara client dan server menggunakan SSL/TLS. Implementasi HTTPS dilakukan melalui konfigurasi reverse proxy pada Nginx sehingga website dapat diakses menggunakan protokol HTTPS.

Penerapan HTTPS bertujuan untuk meningkatkan keamanan komunikasi data serta mengurangi risiko penyadapan pada layanan web yang berjalan di server. Selain itu penggunaan HTTPS juga membantu memastikan bahwa akses website dilakukan melalui koneksi terenkripsi.



Gambar 13 Implementasi HTTPS pada Website WordPress

Berdasarkan hasil implementasi, website WordPress berhasil diakses menggunakan alamat HTTPS pada browser. Hal ini menunjukkan bahwa konfigurasi SSL/TLS pada layanan web berhasil diterapkan dan komunikasi antara client dan server telah menggunakan koneksi terenkripsi. Namun pada browser masih muncul status “Not Secure” karena sertifikat SSL yang digunakan bersifat self-signed atau belum terdaftar pada Certificate Authority (CA). Meskipun demikian, implementasi HTTPS tetap berhasil dilakukan dan dapat digunakan untuk pengujian pada lingkungan server local.

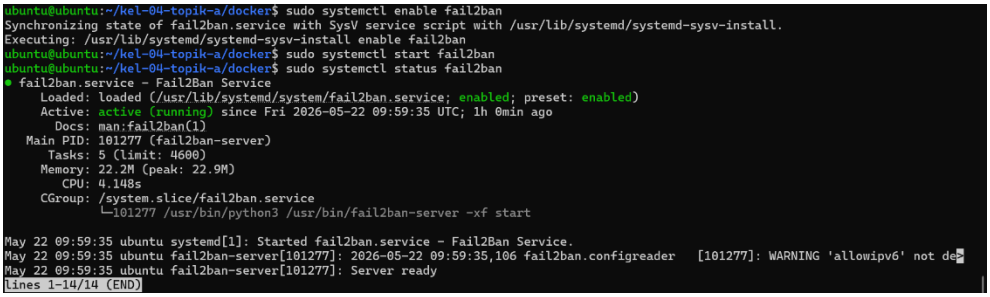
4.8 Implementasi Fail2Ban

Fail2Ban digunakan untuk meningkatkan keamanan server dengan mencegah brute force attack pada layanan SSH. Fail2Ban bekerja dengan memonitor log autentikasi server kemudian memblokir alamat IP yang melakukan percobaan login berulang. Proses instalasi Fail2Ban dilakukan menggunakan perintah berikut:

```
ubuntu@ubuntu:~/kel-04-topik-a$ sudo apt install fail2ban -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-pyasyncore python3-pyinotify whois
```

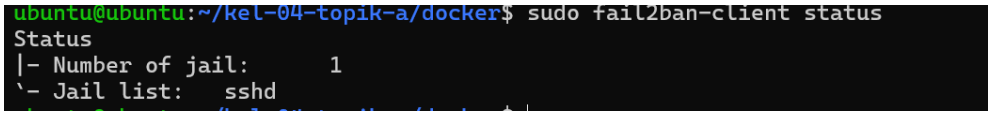
Gambar 14 Instalasi Fail2Ban pada Ubuntu Server

Setelah proses instalasi selesai, service Fail2Ban dijalankan dan diaktifkan pada sistem menggunakan perintah berikut:



Gambar 15 Aktivasi Service Fail2Ban

Untuk memastikan Fail2Ban berjalan dengan baik, dilakukan pengecekan status menggunakan perintah berikut:



Gambar 16 Verifikasi Status Fail2Ban pada Ubuntu Server

Berdasarkan hasil implementasi, Fail2Ban berhasil berjalan pada sistem Ubuntu dan aktif melakukan monitoring terhadap layanan SSH untuk mencegah brute force attack.

4.9 Hasil Running Container

Setelah seluruh konfigurasi Docker Compose selesai dilakukan, tahap selanjutnya adalah melakukan pengecekan status container untuk memastikan seluruh layanan berjalan dengan baik. Pengecekan dilakukan menggunakan perintah berikut:



Gambar 17 Hasil Running Container Docker

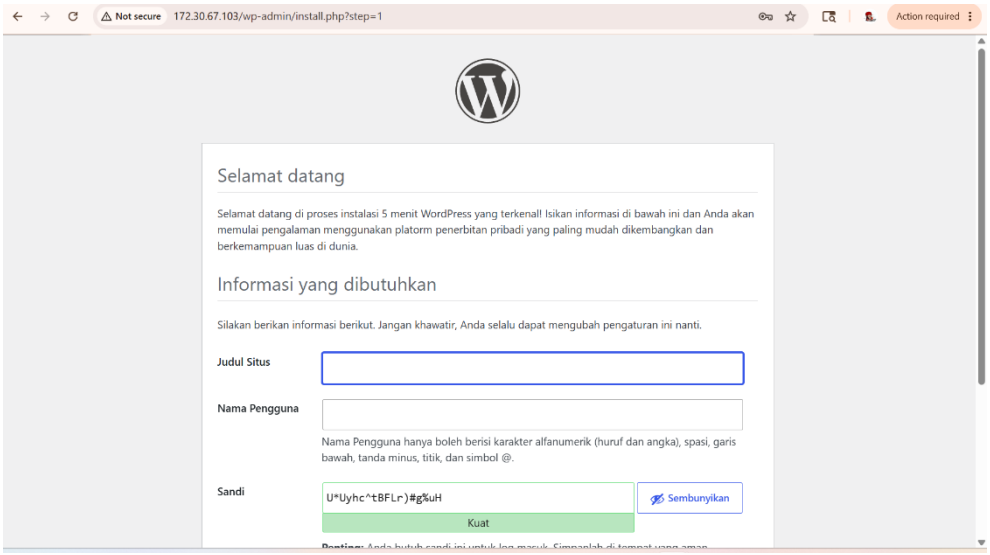
Berdasarkan hasil implementasi, sistem berhasil menjalankan tiga container utama yaitu nginx_server, wordpress_app, dan mariadb_db. Seluruh container berada pada status Up yang menandakan bahwa layanan berhasil berjalan dengan baik tanpa mengalami error pada proses deployment.

Container nginx_server berfungsi sebagai reverse proxy yang menangani request dari client, container wordpress_app digunakan untuk menjalankan aplikasi website WordPress, sedangkan container mariadb_db digunakan sebagai database server untuk menyimpan data

website. Hasil tersebut menunjukkan bahwa implementasi layanan multi-container menggunakan Docker Compose berhasil dilakukan sesuai dengan perancangan sistem yang telah dibuat.

4.10 Implementasi WordPress

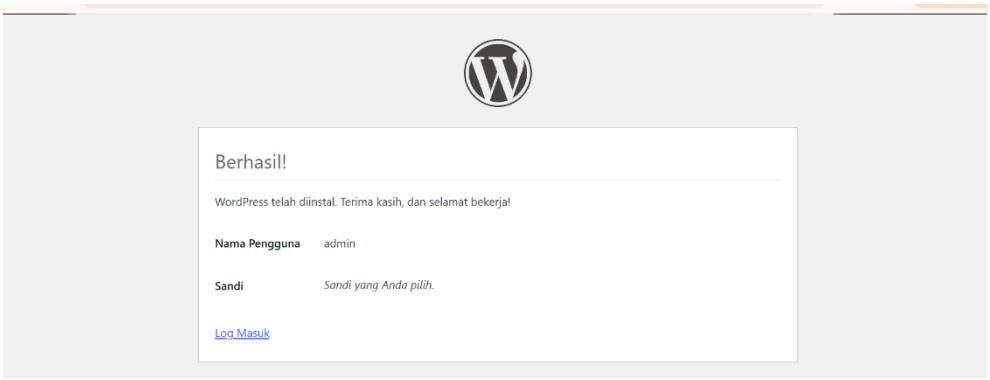
Setelah seluruh container berhasil dijalankan, tahap selanjutnya adalah melakukan implementasi WordPress melalui browser. Website WordPress diakses menggunakan alamat IP server yang telah dikonfigurasi pada layanan Nginx reverse proxy. Pada tahap awal, sistem menampilkan halaman instalasi WordPress untuk melakukan konfigurasi website dan pembuatan akun administrator.



Gambar 18 Halaman Instalasi WordPress

Berdasarkan hasil implementasi, halaman instalasi WordPress berhasil ditampilkan pada browser. Pada tahap ini dilakukan pengisian informasi website seperti judul situs, username administrator, password, dan email yang digunakan untuk proses konfigurasi awal WordPress.

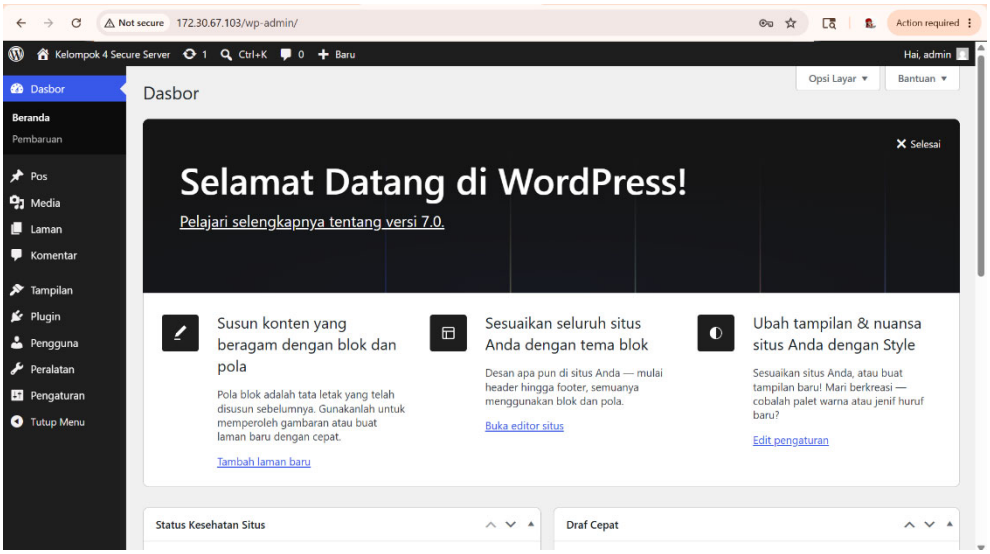
Setelah seluruh informasi berhasil diisi, proses instalasi WordPress berhasil dilakukan dan sistem menampilkan halaman konfirmasi keberhasilan instalasi.



Gambar 19 Hasil Instalasi WordPress Berhasil

Hasil implementasi menunjukkan bahwa WordPress berhasil diinstal pada server dan akun administrator berhasil dibuat. Selanjutnya pengguna dapat melakukan login ke dashboard administrator WordPress untuk mengelola website.

Tahap berikutnya adalah melakukan login ke halaman administrator WordPress menggunakan akun yang telah dibuat sebelumnya.



Gambar 20 Dashboard Administrator WordPress

Berdasarkan hasil pengujian, dashboard administrator WordPress berhasil diakses dengan baik melalui browser. Hal ini menunjukkan bahwa layanan Nginx, WordPress, dan MariaDB berhasil saling terhubung menggunakan Docker container sehingga aplikasi website dapat berjalan sesuai dengan perancangan sistem yang telah dibuat.

BAB V
PENGUJIAN DAN ANALISIS

5.1 Pengujian Sistem

Pengujian sistem dilakukan untuk memastikan seluruh layanan pada secure web server dapat berjalan dengan baik sesuai dengan perancangan sistem yang telah dibuat. Pengujian dilakukan terhadap seluruh layanan utama yang digunakan pada sistem seperti Docker container, WordPress, MariaDB, Nginx reverse proxy, Firewall UFW, dan Fail2Ban. Selain itu pengujian juga dilakukan untuk memastikan komunikasi antar container berjalan dengan baik melalui Docker network.

Proses pengujian dilakukan menggunakan beberapa metode seperti pengecekan status container menggunakan terminal, pengujian akses website melalui browser, pengecekan konfigurasi firewall, serta verifikasi layanan keamanan server. Pengujian ini bertujuan untuk mengetahui apakah seluruh layanan berhasil dijalankan tanpa mengalami error dan dapat bekerja sesuai fungsi masing-masing.

Pengujian	Metode Pengujian	Hasil
Docker Container	docker ps	Berhasil
WordPress	Akses website melalui browser	Berhasil
MariaDB	Koneksi database WordPress	Berhasil
Nginx Reverse Proxy	Akses layanan melalui Nginx	Berhasil
Firewall UFW	ufw status verbose	Berhasil
Fail2Ban	fail2ban-client status	Berhasil

Tabel 2 Pengujian Sistem

Pada pengujian Docker container, seluruh container berhasil berjalan dengan status aktif berdasarkan hasil perintah docker ps. Container yang berhasil dijalankan meliputi container Nginx sebagai reverse proxy, container WordPress sebagai web application, dan container MariaDB sebagai database server. Selain itu WordPress berhasil diakses melalui browser menggunakan alamat IP server dan dapat terhubung dengan database MariaDB sehingga website dapat berjalan dengan baik tanpa mengalami kendala.

Pada pengujian keamanan server, layanan Nginx berhasil menerima request dari client dan meneruskannya ke aplikasi WordPress yang berjalan di dalam container Docker. Website juga berhasil diakses menggunakan protokol HTTP maupun HTTPS sehingga menunjukkan

bahwa konfigurasi reverse proxy dan SSL/TLS berhasil diterapkan. Selain itu Firewall UFW berhasil aktif dengan hanya membuka port tertentu yang dibutuhkan layanan server yaitu port 22, 80, dan 443, sedangkan Fail2Ban berhasil berjalan untuk membantu mencegah brute force attack pada layanan SSH.

5.2 Pengujian Keamanan

Pengujian keamanan dilakukan menggunakan konfigurasi Firewall UFW, SSH Hardening, HTTPS/SSL, dan Fail2Ban untuk memastikan layanan server berjalan dengan aman. Hasil pengujian menunjukkan bahwa:

1. Port yang terbuka hanya port yang dibutuhkan layanan server yaitu port 22, 80, dan 443.
2. Fail2Ban berhasil berjalan dan digunakan untuk memonitor layanan SSH terhadap brute force attack.
3. Website berhasil diakses menggunakan protokol HTTPS sehingga komunikasi antara client dan server telah menggunakan SSL/TLS.
4. Browser masih menampilkan status “Not Secure” karena sertifikat SSL yang digunakan bersifat self-signed atau belum terdaftar pada Certificate Authority (CA).

5.3 Analisis Hasil

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, sistem secure web server berhasil dijalankan menggunakan arsitektur multi-container berbasis Docker. Seluruh layanan utama seperti Nginx reverse proxy, WordPress, dan MariaDB berhasil berjalan dengan baik dan saling terhubung melalui Docker network.

Penggunaan Docker Compose membantu mempermudah proses deployment dan pengelolaan container karena seluruh layanan dapat dijalankan secara otomatis melalui satu file konfigurasi. Selain itu Docker network membantu meningkatkan isolasi layanan sehingga komunikasi antar container menjadi lebih aman dan terstruktur.

Implementasi Firewall UFW dan Fail2Ban juga berhasil meningkatkan keamanan server dengan membatasi akses jaringan dan mencegah brute force attack pada layanan SSH. Website berhasil diakses menggunakan protokol HTTPS sehingga komunikasi data antara client dan server telah menggunakan enkripsi SSL/TLS.

Secara keseluruhan, implementasi secure web server berbasis Docker berhasil dilakukan sesuai tujuan perancangan sistem dan mampu menjalankan layanan web dengan baik pada lingkungan Ubuntu Server.

BAB VI
PENUTUP

6.1 Kesimpulan

Berdasarkan hasil implementasi yang telah dilakukan, sistem secure web server berbasis Docker berhasil dibangun menggunakan arsitektur multi-container yang terdiri dari Nginx sebagai reverse proxy, WordPress sebagai web application, dan MariaDB sebagai database server. Seluruh layanan berhasil dijalankan menggunakan Docker Compose dan saling terhubung melalui Docker network sehingga sistem dapat berjalan dengan baik sesuai perancangan yang telah dibuat.

Selain implementasi layanan web, konfigurasi keamanan server juga berhasil diterapkan menggunakan Firewall UFW, HTTPS/SSL, Fail2Ban, serta konfigurasi layanan SSH untuk meningkatkan keamanan server. Website berhasil diakses menggunakan protokol HTTPS sehingga komunikasi antara client dan server telah menggunakan enkripsi SSL/TLS, meskipun sertifikat yang digunakan masih bersifat self-signed. Penggunaan Docker membantu mempermudah proses deployment dan pengelolaan layanan, sedangkan implementasi keamanan server membantu meningkatkan perlindungan terhadap ancaman jaringan dan akses tidak sah pada server.

6.2 Saran

Berdasarkan hasil implementasi yang telah dilakukan, sistem secure web server masih dapat dikembangkan lebih lanjut untuk meningkatkan keamanan dan performa layanan. Pengembangan yang dapat dilakukan di antaranya adalah menambahkan sistem monitoring server secara realtime untuk memantau penggunaan resource dan aktivitas layanan server secara lebih efektif. Selain itu penggunaan sertifikat SSL resmi dari Certificate Authority (CA) juga dapat diterapkan agar website dapat diakses dengan status secure pada browser tanpa peringatan keamanan.

Pengembangan lainnya yang dapat dilakukan yaitu menambahkan sistem backup otomatis pada database dan volume Docker untuk mengurangi risiko kehilangan data apabila terjadi kerusakan sistem. Selain itu implementasi intrusion detection system (IDS) juga dapat digunakan untuk meningkatkan keamanan server terhadap berbagai ancaman jaringan yang lebih kompleks. Sistem deployment juga dapat dikembangkan lebih lanjut agar layanan dapat berjalan menggunakan domain publik dan lingkungan production server sehingga

implementasi secure web server menjadi lebih optimal dan siap digunakan pada skala yang lebih besar.

DAFTAR PUSTAKA

Adiputra Firmansyah. (2015). Container Dan Docker: Teknik Virtualisasi Dalam Pengelolaan Banyak Aplikasi Web. *Jurnal SimanteC*, *4*(3).

Dwiyatno, S., Rachmat, E., Sari, A. P., & Gustiawan, O. (2020). Implementasi Virtualisasi Server Berbasis Docker Container. *PROSISKO: Jurnal Pengembangan Riset Dan Observasi Sistem Komputer*, *7*(2), 165–175. https://doi.org/10.30656/prosisko.v7i2.2520

Rakhmadi Rahman, Moh. Farel, & Muhammad Dirga Sopan. (2026). Implementasi Hardening Server Linux Untuk Mengurangi Risiko Serangan Siber. *Jurnal Riset Sistem Informasi*, *3*(2), 39–44. https://doi.org/10.69714/c4atnn70